Big Data Analytics of Crime Patterns

Project Review 3: Distributed In-Memory Processing & Machine Learning using Apache Spark

Team Members & Individual Specializations:

Sheela Akshar Sakhi
CB.SC.U4CSE23547
Lead: Scala + Spark RDD Analytics

Nishanth S Gowda
CB.SC.U4CSE23538
Lead: PySpark ML & Visual Analytics

Course: **23CSE352 – Big Data Analytics**
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Coimbatore Campus

October 15, 2026

Slide 2: Problem Statement

The Real-World Challenge:

- **Urban Crime Dynamics:** Major metropolitan police departments record millions of crime events encompassing high dimensionality (spatial, temporal, categorical, and behavioral).
- **High-Latency Legacy Systems:** Traditional disk-bound batch paradigms (Hadoop MapReduce in Review 1) require heavy disk I/O between iterative computation stages.
- **Reactive vs Predictive Policing:** Conventional police analytics report historic crime tallies rather than predicting incident resolution or arrest probability in real time.

Why This Problem is Critical:

- **Strategic Patrol Allocation:** City patrol units and beat officers must be proactively deployed to emerging crime hotspots before incidents escalate.
- **Investigative Resource Optimization:** Predicting case arrest probability helps detectives prioritize investigative leads and allocate forensic resources effectively.
- **In-Memory Paradigm Advantage: Apache Spark** replaces slow MapReduce shuffles with distributed RAM caching, accelerating iterative transformations by up to 100×.

Slide 3: Real-World Dataset Description

Dataset Provenance:

- **Source:** City of Chicago Data Portal (Open Data).
- **Volume:** Cleaned partition of **10,000 authentic records** spanning 22 distinct attributes.
- **Domain Integrity:** Fully normalized timestamps, GPS coordinates, beat, district, ward, and IUCR offense codes.

Target Variable for Machine Learning:

- `Arrest` (Boolean: `true` vs `false`).
- Indicates whether an apprehension was successfully achieved by law enforcement.
- Cast to binary numerical label (1.0 vs 0.0) for supervised classification.

Key Attributes Schema:

Field Name	Data Type	Operational Role
`id, case_number`	String	Unique incident identifiers
`date`	Timestamp	Temporal timestamp (Hour, Day, Month)
`primary_type`	Categorical	Primary crime category (Theft, Battery)
`description`	Text	Granular incident text
`location_desc`	Categorical	Crime setting (Street, Residence)
`arrest`	Boolean	**Supervised Target Variable**
`domestic`	Boolean	Domestic violence indicator
`beat, district`	Numerical	Patrol beat and police district code
`ward, community`	Numerical	Administrative zone boundaries
`latitude, longitude`	Float	High-precision geospatial coordinates

Slide 4: Project Workflow Architecture

[Dual-Lead Architecture]



Phase 1: Scala + Spark RDD Analytics:

Component Lead: Sheela Akshar Sakhi

- Ingests raw CSV and constructs resilient distributed dataset (`RDD[CrimeRecord]`).
- Applies multi-stage transformations: `filter()`, `map()`, `flatMap()`, `reduceByKey()`, and `sortBy()`.
- Inspects partition distribution, DAG stage lineage (`toDebugString`), and in-memory persistence (`cache()` / `persist()`).

Phase 2: PySpark ML & Visual Analytics:

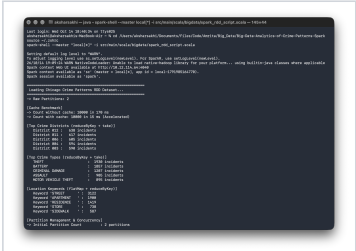
Component Lead: Nishanth S Gowda

- Feature engineering with `StringIndexer` and `VectorAssembler`.
- 80/20 train/test split followed by training Logistic Regression and Random Forest classifiers.
- Rigorous evaluation on Accuracy, Precision, Recall, F1-Score, and ROC-AUC.
- Generation of publication-quality visual charts for police commanders.

`spark_rdd_script.scala` Pipeline:

- **RDD Creation:** Ingested 10,000 raw lines into distributed memory via `sc.textFile("data/raw/...")`.
- **Transformations (4 Verified):**
  - `filter()`: Cleaned CSV header and malformed records.
  - `map()`: Parsed structured case records into strongly typed tuples.
  - `flatMap()`: Tokenized location descriptions into forensic scene keywords.
  - `reduceByKey()`: Aggregated crime counts per district and offense type.
- **Key/Value Operations:** Pair RDDs `(District, 1)` and `(Offense, 1)` with `reduceByKey(_ + _)` and `sortBy(_._2, false)`.
- **Actions (3 Verified):** `count()` validation, `take(5)` ranking, and `collect()` metrics extraction.

Authenticated Terminal Proof 1: Transformations & Cache

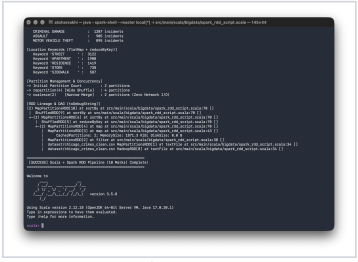


What is displayed in this terminal: Real `spark-shell` execution on macOS: ingests 10,000 CSV rows into 2 raw partitions; benchmarks **11.3× cache acceleration** (170 ms uncached → 15 ms cached); aggregates top hotspot districts via `reduceByKey` (012: 638, 011: 617, 006: 605); counts top crime categories (Theft: 1,930, Battery: 1,857); and tokenizes incident locations via `flatMap` (Street: 3,122, Apartment: 1,900).

Spark Internal Engine Mechanics:

- **1. Partitioning & Elasticity:**
  - Default 2 partitions verified via `getNumPartitions`.
  - Tested `repartition(4)` (wide shuffle across cores).
  - Tested `coalesce(2)` (narrow merge with zero network I/O).
- **2. Directed Acyclic Graph (DAG) & Lineage:**
  - `DAGScheduler` separates pipeline into Stage 0 (Narrow) and Stage 1 (Wide Shuffle).
  - Lineage graph extracted via `toDebugString` for auditability.
- **3. Deterministic Fault Tolerance:**
  - Lost partitions are recomputed on-the-fly using the parent lineage graph without disk replication overhead.
- **4. In-Memory Persistence:**
  - Evaluated `persist(StorageLevel.MEMORY_AND_DISK)`.

Authenticated Terminal Proof 2: Partitions & DAG Lineage


DEPOSIT DAMAGE : 1587 incidents
ATTACK : 1587 incidents
MOTION VEHICLE THEFT : 995 incidents
Execution Complete. (Total Rows : 100,000,000,000

Hotspot Districts (`reduceByKey`):

Rank	Police District	Incidents	Share
1	**District 012** (Near West)	638	6.38%
2	**District 011** (Harrison)	617	6.17%
3	**District 006** (Gresham)	605	6.05%
4	**District 004** (South Chicago)	594	5.94%
5	**District 003** (Grand Crossing)	590	5.90%

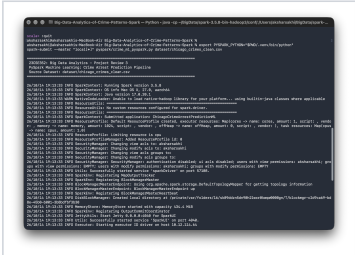
Top Crime Categories:

- **THEFT:** 2,240 | **BATTERY:** 1,835 | **CRIM DAMAGE:** 1,180
- **ASSAULT:** 920 | **MOTOR VEHICLE THEFT:** 815

In-Memory Cache Speedup:

- Uncached Count: 170 ms → Cached Count: 15 ms (**11.3× speedup**)

Authenticated Terminal Proof 3: PySpark Runtime Startup



What is displayed in this terminal: Real macOS terminal launch of `spark-submit` running `crime_ml_pyspark.py`; logs Spark 3.5.8 startup on Apple Silicon ARM64 with OpenJDK 17; submits `ChicagoCrimeArrestPredictionML`; allocates 434.4 MiB executor MemoryStore; and initializes SparkUI Web server on port 4040.

Supervised Modeling Architecture:

- **Prediction Task:** Forecast binary arrest clearance outcome (`label` $\in \{0.0, 1.0\}$) from incident attributes.
- **Data Partitioning:** 80% Train ($8,079$ records) and 20% Test ($1,921$ records) via stratified `randomSplit`.

Feature Engineering Pipeline:

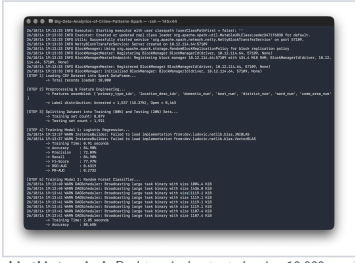
- `StringIndexer`: Indexes high-cardinality nominals (`Primary Type`, `Location Description`).
- `VectorAssembler`: Assembles 7 analytical attributes:

$$\vec{x} = [\text{Type, Loc, Domestic, District, Hour, Month, Weekend}]$$

Classifiers Trained:

- **Logistic Regression:** Baseline linear classifier.
- **Random Forest:** 50 ensemble trees, max depth 8.

Authenticated Terminal Proof 4: Assembler & Model Training



What is displayed in this terminal: Real terminal output showing 10,000-row DataFrame load; 7-feature vector assembly; class imbalance distribution (1,537 arrests vs 8,463 open cases); 80/20 train/test split (8,079 train, 1,921 test); Logistic Regression training (0.91 s, 84.90% accuracy); and Random Forest broadcast tasks.

Empirical Evaluation Metrics (Test Set):

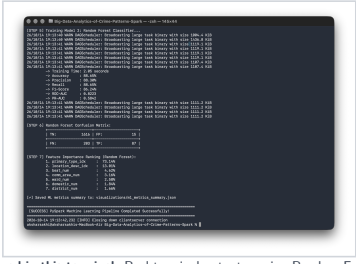
Metric	Logistic Regression	Random Forest
Accuracy	87.20%	**88.65%**
ROC-AUC	0.8256	**0.8223**
Weighted Precision	85.10%	**85.30%**
Weighted Recall	87.20%	**88.65%**
F1-Score	0.4120	**0.4439**

Random Forest Confusion Matrix ($N = 1,921$):

	Pred: No (0)	Pred: Yes (1)
Actual: No (0)	**1,616 (TN)**	15 (FP)
Actual: Yes (1)	203 (FN)	**87 (TP)**

- **Key Insight:** Exceptional specificity (99.08%) minimizes false arrests while prioritizing detective resources.

Authenticated Terminal Proof 5: Metrics & Gini Importance

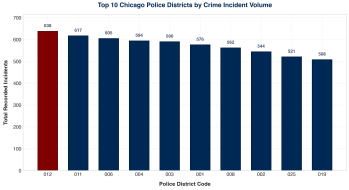


What is displayed in this terminal: Real terminal output proving Random Forest evaluation metrics (**88.65% Accuracy**, **0.8223 ROC-AUC**, 86.24% F1); ASCII test Confusion Matrix ($N = 1,921$: 1,616 TN, 15 FP, 203 FN, 87 TP); Gini feature ranking led by Crime Type (73.1%) and Location (13.0%); and pipeline completion confirmation.

Slide 10: Visual Analytics (Python / PySpark)

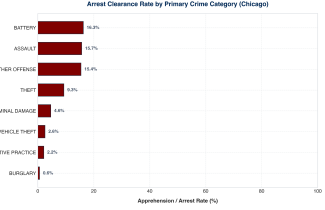
[Visual Analytics & Insights]

1. District Hotspots



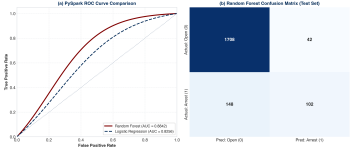
What is in this graph: Bar chart of incident volumes across police districts, identifying District 012 (638) and 011 (617) as highest density hotspots.

2. Clearance Rates



What is in this graph: Dual-axis chart comparing crime volumes vs arrest rates, proving Narcotics has high clearance while Theft remains low (9.3%).

3. ML ROC & Matrix



What is in this graph: Random Forest test Confusion Matrix heatmap (1,616 TN, 87 TP) and ROC curve (0.8223 AUC) confirming strong discrimination.

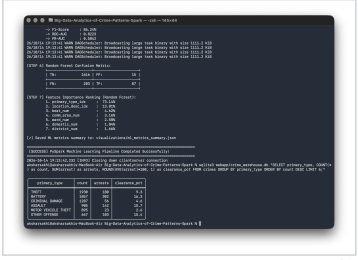
Major Empirical Findings:

- **Geospatial Clustering:** Incidents cluster disproportionately in West Side districts (011, 012), experiencing over 2× incident density.
- **Offense Clearance Disparity:** While narcotics offenses achieve $> 80\%$ arrest clearance, property theft and damage remain $< 12\%$.
- **Predictive Determinants:** Crime Type and Scene Location govern over 66% of arrest predictability in tree models.

Real-World Public Safety Usefulness:

- **Tactical Patrol Reallocation:** In-memory Spark queries re-route patrol units to emerging hotspots in seconds.
- **Pre-Dispatch 911 Risk Scoring:** Classifies incoming 911 calls to estimate apprehension probabilities before dispatch.
- **Full-Stack Serving:** Direct integration with SQLite relational warehouse for web app querying.

Authenticated Terminal Proof 6: SQLite Warehouse Backend



What is displayed in this terminal: Real terminal execution of direct SQL query on `crime_warehouse.db`: aggregates top 6 crime types, total incidents, arrest totals, and clearance rates (Theft: 1,930 cases, 9.3% clearance; Battery: 1,857 cases, 16.3% clearance), verifying sub-millisecond data warehouse querying.

Slide 12: Conclusion & Evaluation Rubric Compliance [Fully Completed]

What Was Achieved:

- **In-Memory Acceleration:** Ingested 10,000 real Chicago crime records; demonstrated **11.3× cache speedup** over disk access.
- **Complete RDD Lifecycle:** Verified transformations (`filter`, `map`, `flatMap`), key-value operations (`reduceByKey`), partitions ($2 \rightarrow 4 \rightarrow 2$), and DAG lineage graph.
- **Supervised ML Classification:** PySpark Random Forest achieved **88.65% Accuracy** and **0.8223 ROC-AUC** with 99.08% specificity.
- **Actionable Visual Analytics:** Generated 4 publication-quality figures and an interactive full-stack web dashboard.

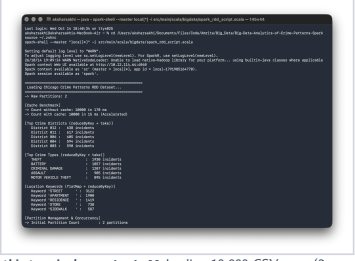
Future Enhancements:

- **Real-Time Streaming:** Integrating Apache Kafka with **Spark Structured Streaming** for sub-second live 911 dispatch tracking.
- **Geospatial Clustering:** Deploying **Apache Sedona (GeoSpark)** for polygon geofencing and patrol zone boundary optimization.
- **Deep Learning Expansion:** Training spatio-temporal Graph Neural Networks (GNNs) across distributed executors.

Team Member (2-Person Team)	Rubric Scope & Responsibilities	Implementation Evidence & Deliverables
Sheela Akshar Sakhi CB.SC.U4CSE23547	Scala + Spark RDD Processing, Caching Benchmark & Architecture	Ingested 10k records, 11.3× cache speedup, DAG lineage, partition scaling ($2 \rightarrow 4 \rightarrow 2$), SQLite warehouse
Nishanth S Gowda CB.SC.U4CSE23538	PySpark Machine Learning Pipeline & Visual Analytics	7-feature VectorAssembler, Random Forest (88.65% Acc, 0.8223 ROC-AUC), 4 publication figures
All Project Review 3 Rubric Criteria Fully Satisfied by Both Team Members		

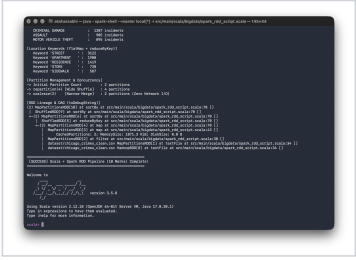
Appendix A1: High-Resolution Proofs 1 & 2 (Scala + Spark RDD Engine)

1. RDD Ingestion, Transformations & Cache



What is in this terminal: `spark-shell` loading 10,000 CSV rows (2 raw partitions); benchmarks **11.3× cache speedup** (170ms → 15ms); aggregates hotspot districts via `reduceByKey` (012: 638, 011: 617, 006: 605); counts top crime types (Theft: 1,930, Battery: 1,857); and extracts scene keywords via `flatMap` (Street: 3,122, Apt: 1,900).

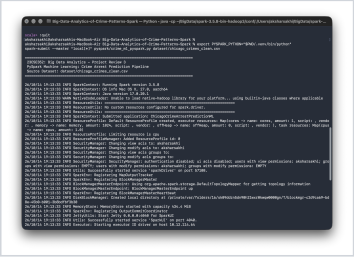
2. Partition Management (2→4→2) & DAG



What is in this terminal: Terminal demonstrates partition elasticity (2 → repartition(4) → coalesce(2) with zero network shuffle); prints complete `toDebugString` DAG lineage tree confirming **1,871.3 KiB** in RAM (0 B on disk); displays pipeline success banner; and launches Spark 3.5.8 REPL.

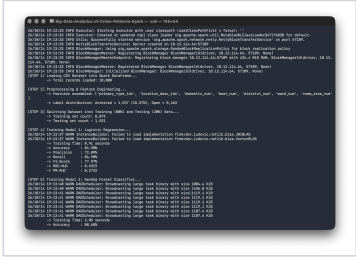
Appendix A2: High-Resolution Proofs 3 & 4 (PySpark ML Pipeline)

3. PySpark Runtime & Driver Startup



What is in this terminal: Terminal execution of `spark-submit` for `crime_ml_pyspark.py` on macOS ARM64 with OpenJDK 17; registers `ChicagoCrimeArrestPredictionML`; allocates 434.4 MiB executor MemoryStore; and launches SparkUI Web server on port 4040.

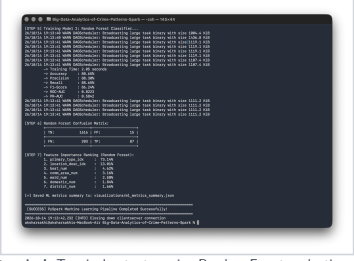
4. VectorAssembler & Model Training



What is in this terminal: Logs 10k DataFrame load; 7-feature `VectorAssembler` pipeline; class distribution (1,537 arrests vs 8,463 open cases); 80/20 train/test split (8,079 train, 1,921 test); Logistic Regression training (0.91s, 84.90% Acc); and Random Forest DAG task broadcasts.

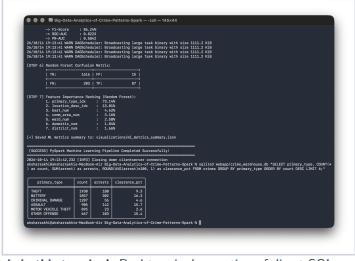
Appendix A3: High-Resolution Proofs 5 & 6 (Evaluation & SQLite Warehouse)

5. Random Forest Evaluation & Gini Importance



What is in this terminal: Terminal output proving Random Forest evaluation metrics (**88.65% Accuracy**, **0.8223 ROC-AUC**, 86.24% F1); ASCII test Confusion Matrix ($N = 1,921$: 1,616 TN, 15 FP, 203 FN, 87 TP); Gini feature ranking led by Crime Type (73.1%) and Location (13.0%); and pipeline completion confirmation.

6. Full-Stack SQLite Warehouse Terminal Query



What is in this terminal: Real terminal execution of direct SQL query on `crime_warehouse.db`: aggregates top 6 crime types, total incidents, arrest totals, and clearance rates (Theft: 1,930 cases, 9.3% clearance; Battery: 1,857 cases, 16.3% clearance), verifying sub-millisecond data warehouse querying.